

5. Project Development Phase

Project: PocketSmart AI - Smart Budget & Recommendation Assistant

5.1 Introduction

The Project Development Phase is the stage in which the planned PocketSmart AI design is converted into an actual working application. The project workflow divides development into Gemini AI setup, core functionality development, backend implementation, user-interface development, and testing and optimization. The purpose of this phase is to implement the required features and connect the different components into one system.

5.2 Development Environment

The project uses Python-based backend development with FastAPI. The frontend uses HTML, CSS, JavaScript, and Jinja2 templates. Gemini AI is integrated to process user inputs and generate recommendations. Environment variables are loaded so that configuration and API access can be managed outside the main application code.

5.3 AI Integration Development

The first implementation activity is to configure access to Gemini and validate API connectivity. The project workflow specifies setting up API access, configuring API keys and usage limits, and making sample prompt calls. Both text-only and image-plus-text inputs are tested to support multimodal functionality.

5.4 Prompt Development

Planner-specific prompts are developed to help the AI interpret budget information, preferences, contextual requirements, and the selected domain. Prompt templates are prepared for Home, Party, and Jewelry planning so that the recommendation process is organized according to the user's selected task.

5.5 Development Objective

The overall development objective is to create a modular recommendation application in which the frontend collects information, the FastAPI backend routes the request, the AI layer generates suggestions, and the results are returned to the user in a clear format.

Page 2 - Backend Development

5.6 Backend Structure

The backend is developed as a modular service layer. The project documentation identifies routes for endpoints, services for recommendation logic and AI calls, and models for input and output schemas. This separation helps keep the application organized and makes individual functions easier to maintain.

5.7 FastAPI Initialization

FastAPI is initialized with the libraries required by the application. Environment configuration is loaded before the main functionality is used. The backend also includes CORS support so that frontend communication can be handled when the application architecture requires cross-origin requests.

5.8 Home Planner Development

The /generate-home route is developed to receive home-related information and generate home interior recommendations. The functionality covers categories such as furniture, decor, and lighting. The recommendation service can use user budget, room details, quantities, style, and related requirements as input.

5.9 Party Planner Development

The /generate-party route is developed to process party details such as event type, guest count, total budget, and venue information. The recommendation logic supports suggestions for catering, decoration, entertainment, and related services.

5.10 Jewelry Planner Development

The /generate-jewelry route is developed for jewelry recommendations. It accepts text-based information and can also process an optional outfit image. The image input supports the planned multimodal use case in which color coordination and aesthetics can be considered by the AI.

5.11 Recommendation Utility Development

The project documentation identifies `gemini_utils.py` as a common utility area for prompt orchestration, budget formatting, domain segmentation, and image analysis. Centralizing these functions supports reuse across planner modules.

Page 3 - Authentication and Session Development

5.12 Registration Development

The /register route is implemented to allow new users to create an account. User credentials are accepted and stored securely so that the application can provide personalized access to its planning and recommendation functions.

5.13 Login Development

The /login route authenticates registered users. After successful authentication, the application can establish the session required for accessing protected planner pages and personalized user functions.

5.14 Logout Development

The /logout route terminates the active user session and redirects the user to the login page. This provides a complete basic authentication flow together with registration and login.

5.15 Token Development

The /token endpoint is developed to issue a secure JWT token after successful authentication. The token is used to support authorized access to protected API routes.

5.16 Session Information

The /session-info endpoint retrieves metadata about the current session, including information such as user ID and login status. The /session-data endpoint provides detailed session-specific information used for personalization and recommendation tracking.

5.17 Recommendation Details and History

The /recommendations-details route is developed to provide detailed AI-generated recommendations based on the user's budget, preferences, and selected category. The /history route retrieves previous recommendation queries and results so users can review or reuse earlier plans.

5.18 Application Startup

The /startup functionality initializes essential application services and configuration when the FastAPI server starts. The main application entry point starts the server using Uvicorn when the script is run directly.

Page 4 - Frontend Development

5.19 Frontend Implementation

The frontend is developed as a lightweight and responsive interface using HTML, CSS, JavaScript, and Jinja2 templates. Its purpose is to collect user information, send requests to the backend, and display AI-generated recommendations in a structured format.

5.20 Home Page

The Home Page acts as the main landing page of PocketSmart AI. It introduces the system's major features and provides links or actions through which users can access the planner modules and user functions.

5.21 Authentication Pages

Register and Login pages are developed to support user account creation and authentication. These pages are connected to the corresponding FastAPI routes so users can access personalized features.

5.22 Planner Pages

Individual planner pages are created for Home Interior, Party, and Jewelry planning. The forms are designed with fields that match each planner's requirements. The Jewelry page also supports optional outfit image upload.

5.23 Recommendation Pages

Separate recommendation views are developed to display the results generated by the AI. The project documentation specifies structured, card-like layouts that can show product suggestions, item details, prices, and other relevant information.

5.24 Dashboard and History

The User Dashboard is developed to show recent recommendations, saved queries, and personalized insights. The Recommendation History page provides access to past queries and results, supporting review and reuse.

5.25 Dynamic Result Display

JavaScript is used to support dynamic result handling and formatting. This allows AI responses to be displayed in a more organized and interactive way rather than presenting raw backend output.

Page 5 - Integration, Testing, and Final Development Output

5.26 Frontend-Backend Integration

After the individual components are developed, the frontend forms are connected to FastAPI POST routes. User inputs are sent to the correct planner endpoint, processed by the recommendation logic, and returned to the frontend for display.

5.27 AI and Application Integration

The Gemini AI layer is integrated into the backend so that planner-specific requests can be transformed into prompts and processed by the model. The returned output is then formatted for the appropriate recommendation page.

5.28 External Platform Integration

The project includes recommendation sourcing from platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, and OYO. During development, the documented workflow allows mock API calls or simulated sourcing to represent these external data sources when required.

5.29 Testing During Development

The application is tested using real-world use cases across Home Interior, Party, and Jewelry planners. Testing covers different budgets, user preferences, event details, product categories, and image inputs where applicable.

5.30 Optimization and Error Handling

The development process includes prompt optimization, input validation, and fallback or default recommendations when the AI produces insufficient results. These measures help reduce input-related failures and improve the reliability of the recommendation system.

5.31 Final Development Output

The completed development phase produces an integrated PocketSmart AI application containing the FastAPI backend, Gemini AI recommendation layer, three planner modules, authentication and session functions, responsive frontend pages, recommendation displays, and history-related functionality.

5.32 Conclusion

The Project Development Phase transforms the PocketSmart AI design into an implemented system. The development process connects backend routes, AI services, planner modules, authentication, frontend interfaces, and testing activities into one working application structure. This phase establishes the functional foundation needed for final validation and use.